SQL

```sql
DROP TABLE employee;

CREATE TABLE employee(
    id INT,
    name VARCHAR(100),
    age INT,
    department VARCHAR(100)
)

RENAME TABLE employee to user

RENAME TABLE user to employee

-- CURD => CREATE READ UPDATE DELETE

ALTER TABLE employee ADD email VARCHAR(100)

ALTER TABLE employee MODIFY name VARCHAR(50)

ALTER TABLE employee DROP COLUMN email;

ALTER TABLE employee CHANGE COLUMN department dept VARCHAR(100)
```

workbench

CREATE DATABASE demo1

show databases;

DROP DATABASE demo1;

use demo

show tables;

CREATE TABLE employee(

id INT,

name VARCHAR(100),

age INT,

department VARCHAR(50)

)

Database, DBMS, RDBMS

What is a Database?

A **Database** is an organized collection of related data stored electronically in a structured format so that it can be easily accessed, managed, and updated.

Real-Time Uses of Database:

- Banking systems – customer accounts, transactions
- Educational institutions – student records, marks
- Hospitals – patient details, medical history
- E-commerce – products, orders, payments
- Social media – user profiles, posts, messages

What is DBMS?

DBMS (Database Management System) is software that allows users to create, store, retrieve, update, and manage databases efficiently.

Functions of DBMS:

- Data storage and retrieval
- Data security and access control
- Backup and recovery
- Data consistency and integrity

Types of DBMS:

- **Hierarchical DBMS** – Tree structure (e.g., IBM IMS)
- **Network DBMS** – Graph structure
- **Relational DBMS** – Table-based structure
- **Object-Oriented DBMS** – Objects as data

Examples: MS Access, File System

What is RDBMS?

RDBMS (Relational Database Management System) is a type of DBMS where data is stored in the form of tables consisting of rows and columns, and relationships are maintained using keys.

Key Features of RDBMS:

- Data stored in tables
- Uses **Primary Key** and **Foreign Key**
- Supports SQL
- Follows ACID properties

- Ensures data integrity and consistency

Examples of RDBMS:

- MySQL
- Oracle
- Microsoft SQL Server
- PostgreSQL
- IBM DB2

DBMS vs RDBMS

DBMS	RDBMS
Stores data as files	Stores data in tables
No relationship between data	Uses relationships with keys
Less security	High security
Does not support SQL fully	Fully supports SQL
Examples: File System, MS Access	Examples: MySQL, Oracle

What is SQL?

SQL (Structured Query Language) is a standard programming language used to interact with relational databases.

- Used to store, retrieve, update, and delete data
- Works with all RDBMS systems
- Standardized by **ANSI (1986)** and **ISO (1987)**

Uses of SQL

- Create databases and tables
- Insert new records
- Update existing data
- Delete records
- Retrieve data using queries
- Apply constraints and security
- Perform data analysis and reporting

What is MySQL?

Definition

MySQL is an **open-source Relational Database Management System (RDBMS)**that stores data in the form of **tables (rows and columns)** and uses **SQL (Structured Query Language)** to manage and manipulate data.

Key Features of MySQL

- Relational Database Management System (RDBMS)
- Open-source and free to use
- Uses SQL for database operations
- High performance and fast data processing
- Reliable, secure, and scalable
- Supports large databases and high traffic
- Cross-platform (Windows, Linux, macOS)
- ANSI SQL compliant
- First released in 1995

Why Use MySQL?

- Widely used in web applications
- Easy to integrate with backend languages like PHP, Java, Python, Node.js
- Strong community support
- Used in real-world applications like e-commerce, banking, and ERP systems

MySQL Data Types

SQL Data Types define the type of data that can be stored in a table column. Choosing the correct data type improves performance, accuracy, and storage efficiency.

Numeric Data Types

- **TINYINT** – 1 byte (8 bits), range: -128 to 127
- **SMALLINT** – 2 bytes (16 bits), range: -32,768 to 32,767
- **INT** – 4 bytes (32 bits), range: -2,147,483,648 to 2,147,483,647
- **BIGINT** – 8 bytes (64 bits), very large integers
- **DECIMAL(M,D)** – Exact numeric value (used for money)
- **FLOAT** – Approximate values (up to ~7 digits)
- **DOUBLE** – Approximate values (up to ~15 digits)

Character / String Types

- **CHAR(n)** – Fixed-length string (1–255 characters)
- **VARCHAR(n)** – Variable-length string (0–65,535 characters)
- **TEXT** – Large text data (articles, descriptions)
- **ENUM** – One value from predefined list

Date & Time Data Types

- **DATE** – YYYY-MM-DD (e.g., 2025-10-06)
- **TIME** – HH:MM:SS
- **DATETIME** – YYYY-MM-DD HH:MM:SS
- **TIMESTAMP** – Auto-updated date & time
- **YEAR** – Year value (YYYY)

Other / Special Data Types

- **BOOLEAN** – TRUE / FALSE
- **BLOB** – Binary data (images, files)
- **JSON** – Stores JSON formatted data

Important Notes

- INT range formula: **-2^{n-1} to $+2^{n-1} - 1$**
- Use **DECIMAL** for financial data (exact precision)
- **FLOAT / DOUBLE** may cause rounding issues
- **TIMESTAMP** is timezone-aware; **DATETIME** is not
- Choose smallest data type possible for better performance

SQL Operators

Arithmetic Operators

- **+** - Addition

SELECT 10 + 5 AS Total;Result: 15

- **-** - Subtraction

SELECT 20 - 8 AS Difference;Result: 12

- ***** - Multiplication

SELECT 6 * 7 AS Product;Result: 42

- **/** - Division

SELECT 15 / 3 AS Quotient;Result: 5

- **%** - Modulo

SELECT 17 % 5 AS Remainder;Result: 2

Comparison Operators

- **=** - Equal to

SELECT * FROM students WHERE marks = 85;

- **!= or <>** - Not equal to

SELECT * FROM students WHERE dept != 'IT';

- **>** - Greater than

SELECT * FROM students WHERE marks > 80;

- **<** - Less than

SELECT * FROM students WHERE marks < 40;

- **>=** - Greater than or equal to

SELECT * FROM students WHERE marks >= 80;

- **<=** - Less than or equal to

SELECT * FROM products WHERE price <= 100;

Logical Operators

- **AND** - Both conditions must be true

SELECT * FROM students WHERE dept = 'IT' AND marks > 60;

- **OR** - Either condition must be true

SELECT * FROM students WHERE dept = 'CS' OR dept = 'IT';

- **NOT** - Negates a condition

SELECT * FROM students WHERE NOT dept = 'Arts';

Special Operators

- **BETWEEN** - Range selection

SELECT * FROM students WHERE marks BETWEEN 50 AND 90;

- **IN** - Match any value in a list

SELECT * FROM students WHERE dept IN ('CS', 'IT', 'ECE');

- **LIKE** - Pattern matching

SELECT * FROM students WHERE name LIKE 'A%';

- **IS NULL** - Check for null values

SELECT * FROM students WHERE email IS NULL;

Assignment Operator

- **=** - Used to assign values in UPDATE statements

```
UPDATE students SET marks = 85 WHERE student_id = 101;

UPDATE employees SET salary = salary * 1.1 WHERE department = 'Engineering';
```